Finding Short Routes with Simulated Annealing and a Genetic Algorithm

CSCI 580 — Assignment 4 · Technical Report

CJ Boyle · October 2026

Summary

This report solves a 40-city Traveling Salesman Problem with two search methods: **Simulated Annealing (SA)** and a **Genetic Algorithm (GA)**. Both start from a simple "go to the closest city" route that is 6.0214 long.

Method	Route length	How much shorter
Starting route (nearest neighbor)	6.0214	—
Simulated Annealing	**5.0057**	**16.87%**
Genetic Algorithm	5.3672	10.86%

Both methods beat the required 10% improvement. **SA worked better:** it found shorter routes, gave similar results every time it ran, and was about 19 times faster. The GA got stuck because its population became copies of each other too early. For extra credit, one small change to the GA (a gentler mutation) made it as good as SA.

1. The Problem

The **Traveling Salesman Problem (TSP)** asks: given a list of cities, what is the shortest route that visits every city once and returns to the start? Think of a delivery driver planning the order of their stops.

It is easy to describe but very hard to solve exactly. With 40 cities there are about 10^{46} possible routes, far too many to check one by one. So instead of searching for the perfect answer, we use **heuristics**: methods that find a very good answer in a reasonable amount of time. TSP-style problems show up in delivery routing, scheduling, and circuit board manufacturing.

The data. The notebook places 40 cities at random spots in a 1 × 1 square. The random seed is fixed, so the cities are the same every run. A route is just a list of the cities in the order we visit them, and its score is the total distance traveled. **Lower is better.**

The starting point. The "nearest neighbor" route always walks to the closest city it hasn't visited yet. It's quick, but it makes messy choices, like the long jumps and crossing lines in Figure 1. Its length, 6.0214, is the baseline we try to beat.

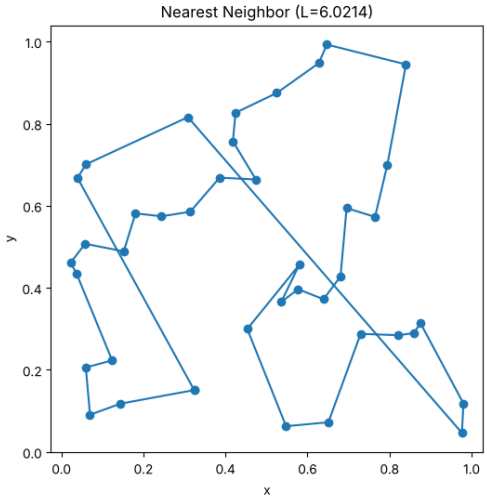


Figure 1. The nearest-neighbor starting route (length 6.0214). Notice the crossing lines and long jumps.

2. How the Methods Work

Both methods use one key move called **2-opt**: pick a section of the route and flip it backwards. For example, A-B-C-D-E with B-C-D flipped becomes A-D-C-B-E. This is good at untangling places where the route crosses itself.

2.1 Simulated Annealing (SA)

The name comes from metalworking: hot metal can be reshaped easily, and as it cools slowly it settles into a strong shape. SA keeps one route and makes 20,000 small 2-opt changes:

- If the change makes the route **shorter**, keep it.
- If it makes the route **longer**, sometimes keep it anyway. The chance is high when the "temperature" is high and close to zero when it's low.
- After every step, lower the temperature a little.

Why accept worse routes? If you only ever accept improvements, you can get stuck on a route where no single small change helps, even though a much better route exists. This is called a **local minimum**. Taking a few steps backwards early on lets SA escape. The two pieces of code I wrote:

```
# Rule for keeping a change
if delta <= 0:                      # new route is shorter (or the same)
    accept = True
else:                               # new route is longer: keep it sometimes
    accept = rng.random() < math.exp(-delta / T)

# Cooling: shrink the temperature a little every step
T = max(T * cfg.alpha, 1e-12)       # max(...) stops T from reaching zero
```

Here `delta` is the new length minus the current length, and `exp(-delta / T)` is the chance of keeping a worse route. It's bigger when T is high or when the route is only a little worse.

2.2 Genetic Algorithm (GA)

The GA copies evolution. It keeps a **population** of 200 routes (the nearest-neighbor route plus 199 random ones) and improves them over 400 **generations**. Each generation:

- **Elitism:** the 5 best routes are copied straight into the next generation.

- **Selection:** pick parents using a tournament. Grab 5 random routes and the shortest one wins.
- **Crossover (90% of the time):** combine two parents into a child route.
- **Mutation (20% of the time):** swap two random cities in the child.

```
def tournament_select(pop, lengths, k, rng):
    contenders = rng.sample(range(len(pop)), k)          # pick k random routes
    winner = min(contenders, key=lambda i: lengths[i])   # shortest one wins
    return pop[winner][:]

def order_crossover_ox(parent1, parent2, rng):
    n = len(parent1)
    a, b = sorted(rng.sample(range(n), 2))
    child = [None] * n
    child[a:b + 1] = parent1[a:b + 1]                    # 1) copy a section of parent 1
    used = set(parent1[a:b + 1])
    fill = [c for c in parent2[b + 1:] + parent2[:b + 1] if c not in used]
    positions = list(range(b + 1, n)) + list(range(0, a))
    for pos, c in zip(positions, fill):                  # 2) fill the gaps with the remaining
        child[pos] = c                                   #    cities in parent 2's order
    return child

def mutate_swap(tour, rng):
    t = tour[:]
    i, j = rng.sample(range(len(t)), 2)                  # pick two positions
    t[i], t[j] = t[j], t[i]                              # swap them
    return t
```

Order crossover needs a little care because a route must contain every city exactly once. Simply gluing half of one parent to half of the other would repeat some cities and skip others. Instead, the child copies one section from parent 1, then fills the empty spots with the missing cities in the order they appear in parent 2.

I also added an optional `stats` argument to both methods. It records extra numbers during the run (such as which changes SA kept) so I could make the graphs in Section 4. It does not change how either method works.

3. Settings and Results

Method	Settings
SA	20,000 steps · starting temperature 0.2 · cooling rate 0.9995 · seed 123
GA	200 routes · 400 generations · tournament size 5 · crossover 90% · mutation 20% · 5 elites · seed 999

Method	Route length	Improvement	Routes checked	Time
Nearest neighbor	6.0214	—	1	—
Simulated Annealing	**5.0057**	**16.87%**	**20,001**	**0.18 s**
Genetic Algorithm	5.3672	10.86%	80,200	3.43 s

"Routes checked" counts how many times each method measured a route's length. It's a fair way to compare how much work each method did.

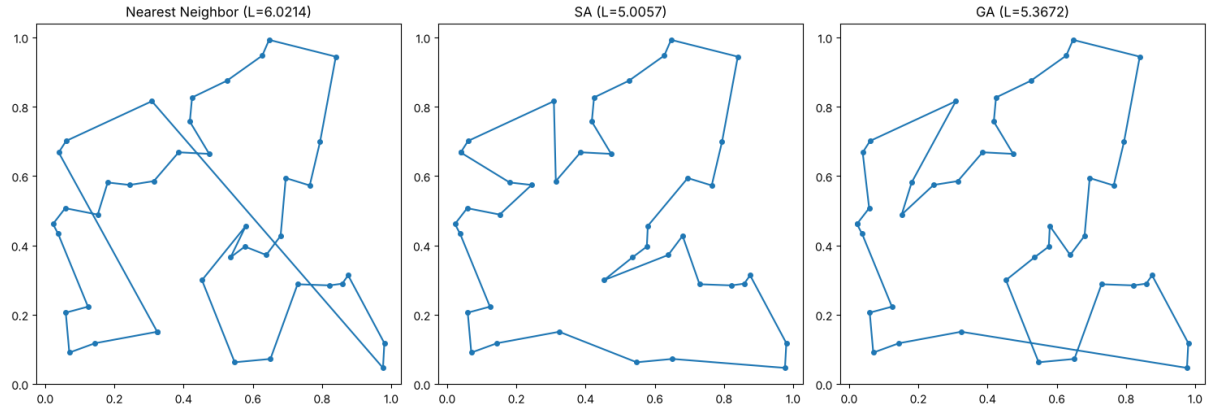


Figure 2. Final routes. SA's route has no crossing lines. The GA's still has some detours left over from the starting route.

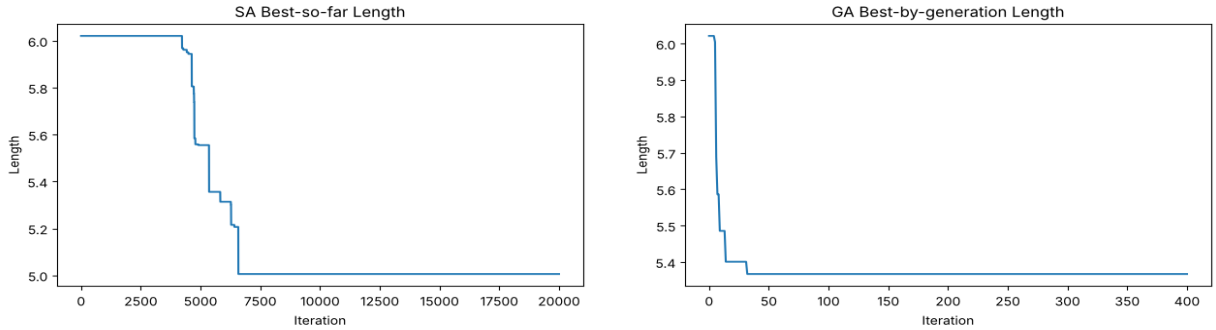


Figure 3. Best route length so far: SA by step (left) and GA by generation (right).

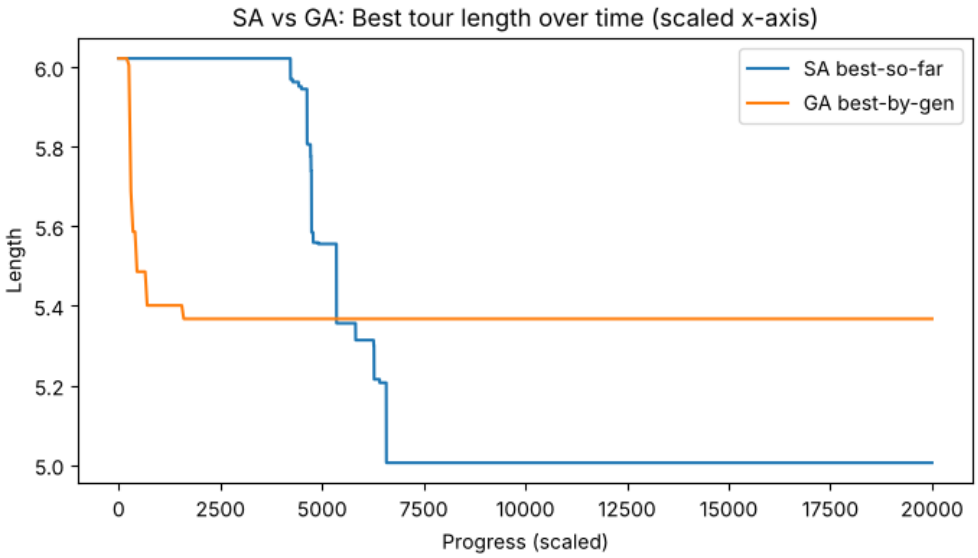


Figure 4. Both progress curves on one graph, stretched to the same width.

4. Looking Inside the Algorithms

4.1 Which changes did SA keep?

SA tried 20,000 changes and kept only 608 of them (3%). Of those, 324 made the route shorter and **283 made it longer on purpose**. Figure 5 shows that the worse changes it kept were mostly only *slightly* worse. Big jumps in the wrong direction were almost always rejected, which is exactly what the rule is designed to do.

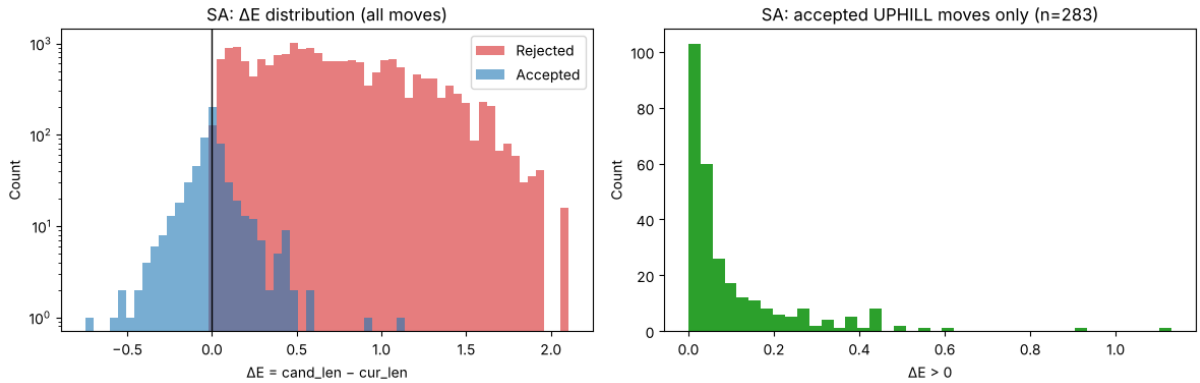


Figure 5. How much each change altered the route length. Left: kept (blue) vs rejected (red). Right: only the worse changes that were kept.

4.2 SA wanders, then settles

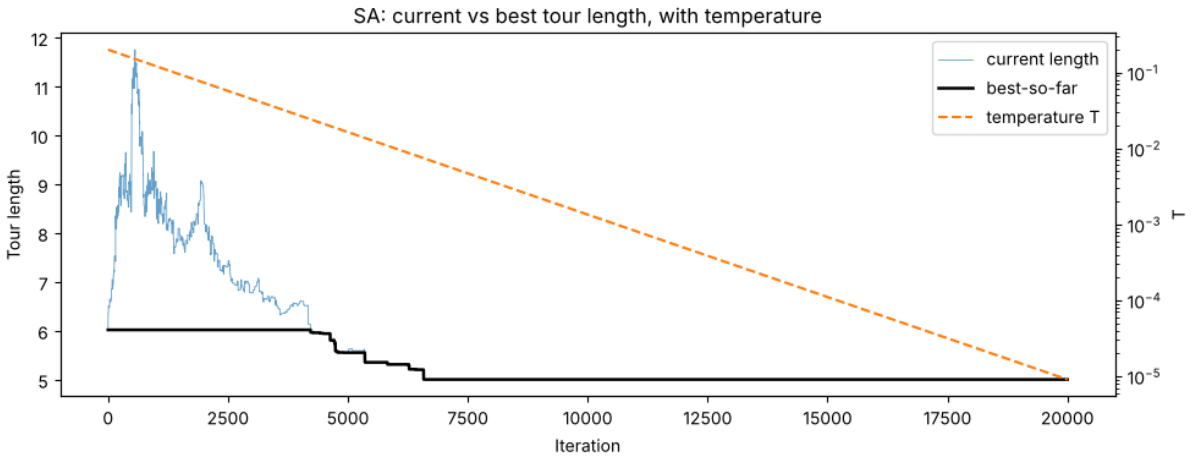


Figure 6. SA's current route (blue), best route so far (black), and temperature (orange dashed line).

While it was hot, SA's current route got much *worse*, climbing up to 11.7. That's SA exploring. As it cooled, it settled down, passed the starting length around step 4,200, and found its best route (5.0057) by step 6,623. After that the temperature was so low that it stopped accepting any worse changes.

4.3 The GA's population

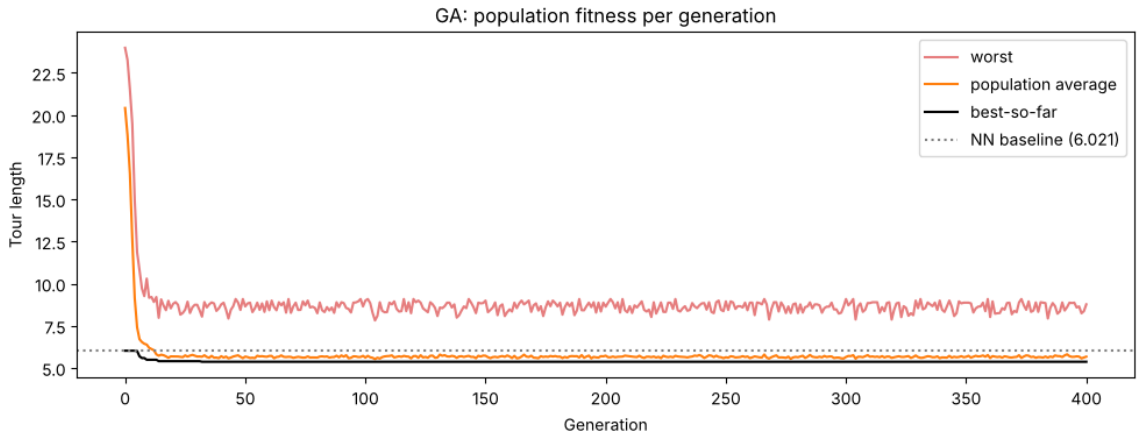


Figure 7. GA: worst route, average route, and best route in each generation.

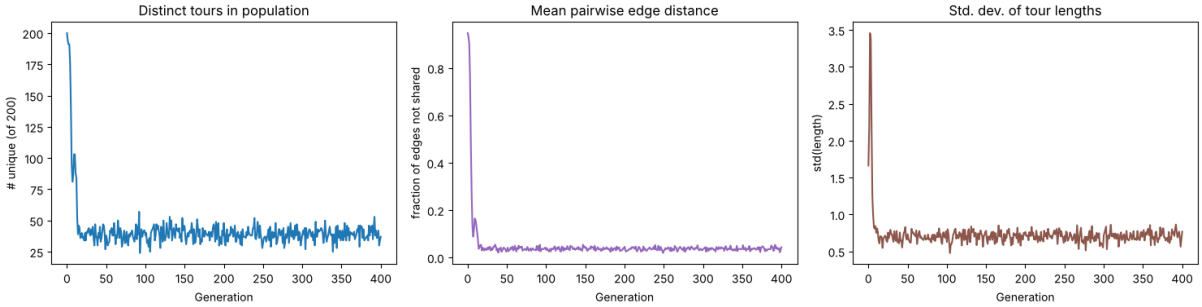


Figure 8. How different the GA's routes are from each other over time.

Generation	0	25	50	100	200	400
Different routes (out of 200)	200	41	38	38	48	37
How different (0 to 1)	0.947	0.042	0.029	0.033	0.046	0.043
Average length	20.416	5.743	5.673	5.611	5.728	5.669
Best length	6.021	5.401	5.367	5.367	5.367	5.367

4.4 Fair comparison and repeat runs

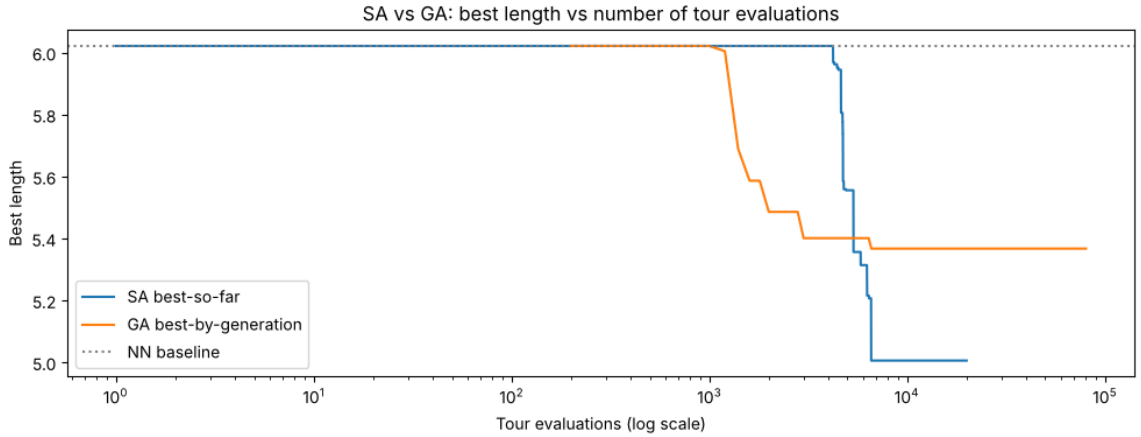


Figure 9. Best length vs. number of routes checked, so both methods are compared for the same amount of work.

Random methods can get lucky or unlucky, so I ran each one 10 more times with different random seeds.

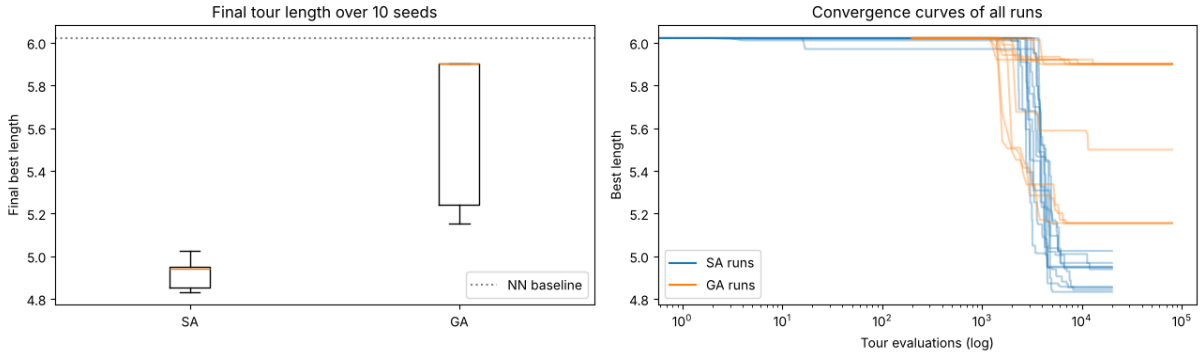


Figure 10. Ten runs of each method: final lengths (left) and every run's progress (right).

10 runs	Average	Spread (std)	Best	Worst
SA	4.9162	0.0618	4.8326	5.0248
GA	5.6363	0.3365	5.1546	5.9001

4.5 Answers to the Part 6 questions

Which algorithm converged faster? The GA *stopped improving* sooner, after only about 15 generations. But it stopped at a worse route, so it got stuck early rather than finishing early. For the same amount of work, SA got better results faster: it beat the GA's final answer after checking only about 5,355 routes, which is around 1/15 of what the GA used.

Which showed more variability? The GA. Across 10 runs its results were spread more than 5 times wider than SA's (0.34 vs 0.06). Some GA runs barely improved on the starting route. SA gave nearly the same answer every time. Within a single run, though, SA's current route jumps around much more, because it explores on purpose.

Did SA escape local minima? Evidence? Yes, for three reasons. (1) It kept 283 changes that made the route worse. (2) Figure 6 shows it leaving the starting route, getting worse, and then coming back down to a much better route. (3) I ran the same search with the temperature set to zero, so it could only accept improvements. That version got stuck at an average of **5.293**, while SA averaged **4.957**. Accepting worse changes is what let SA get past those stuck points.

Did the GA lose diversity? Evidence? Yes, very quickly. By generation 25, the 200 routes had shrunk to only 41 different ones, and any two routes shared about 96% of their connections (Figure 8). After generation 50 the best route never improved again. The reason is that the starting route was about 3 times shorter than the random routes, so its "children" took over the population. Once every route is nearly the same, crossover just creates more copies, and swap mutation usually makes routes worse.

Which worked better, and why? SA. It found shorter routes, was more consistent, and was much faster. Its 2-opt move fits this problem well because it untangles crossings directly. The GA's swap mutation is clumsier: swapping two cities breaks up to four connections at once.

5. Extra Credit (Part 7)

5.1 Trying different GA operators

I tried three changes to the GA, running each 5 times:

- **Inversion mutation:** instead of swapping two cities, flip a section of the route (the same 2-opt move SA uses).
- **PMX crossover:** a different way to combine two parents that keeps cities in the same *positions* instead of the same *order*.
- **Memetic GA:** every generation, polish the best route with quick 2-opt fixes.

```
def mutate_inversion(tour, rng):
    return random_two_opt_neighbor(tour, rng)   # flip a random section
```

GA version (5 runs each)	Average	Spread	Best
Original: crossover + swap	5.647	0.316	5.168
PMX + swap	5.900	0.000	5.900
Crossover + inversion	**4.926**	**0.079**	**4.833**
PMX + inversion	5.129	0.091	5.042
Crossover + swap, no starting route	5.759	0.379	5.200
Crossover + inversion, no starting route	5.060	0.138	4.833
Memetic (crossover + inversion + polish)	5.305	0.142	5.131
SA, for comparison (10 runs)	4.916	0.062	4.833

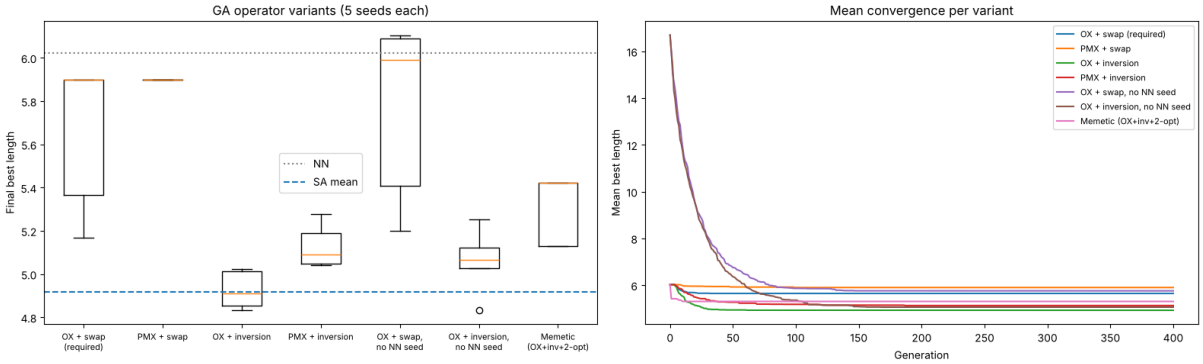


Figure 11. The GA versions compared: final lengths (left) and average progress (right).

- **The mutation made the biggest difference.** Switching from swap to inversion improved the average by about 13% and made the GA as good as SA. Flipping a section only breaks 2 connections, so it doesn't wreck good parts of the route.
- **Order crossover beat PMX.** In a route, what matters is which cities are *next to each other*, not their exact positions. PMX + swap got stuck on the exact same route in all 5 runs.
- **The starting route helps, but it also makes the GA get stuck sooner.** Without it, the GA still did well, so the GA is really doing the work.
- **Polishing didn't help.** The polished route became so much better than the others that it took over the population almost immediately, and diversity disappeared even faster.

5.2 Testing SA's settings

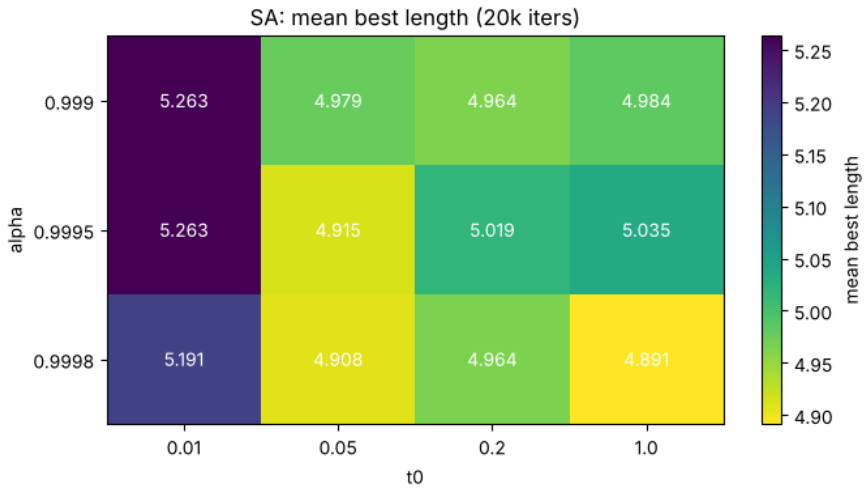


Figure 12. SA's average result for different starting temperatures (t0) and cooling rates (alpha). Lighter = better.

If the starting temperature is too low (0.01), SA acts like a greedy search and gets stuck around 5.2. For any starting temperature of 0.05 or higher, the results were all between 4.89 and 5.04. So SA isn't very picky about its settings, as long as it starts warm enough to explore.

6. Conclusions

- Both methods clearly beat the starting route: SA by 16.9% and the GA by 10.9%.
- SA was the better method here. It was more accurate, more consistent, and faster. Accepting some worse changes on purpose is what let it avoid getting stuck.
- The GA lost its variety within about 25 generations, and after that it stopped improving.
- Choosing operators that fit the problem matters a lot. One smarter mutation made the GA as good as SA.

How to reproduce

Every number and graph in this report comes from `TSP_SA_and_GA_Assignment.ipynb` in the Assignment_4 folder of my GitHub repository. Running all the cells (about 75 seconds) gives the same results, because every run uses a fixed random seed.